ARI711S Assignment – Informed Search

In this notebook, we implement and compare:

- Greedy Best-First Search
- A* Search

We solve a warehouse navigation problem from A to B.

In [1]:
```
# Import required modules
from warehouse import Warehouse
from algorithms import greedy_search, astar_search, reconstruct_path

# Here we load the warehouse map from file
problem = Warehouse("maps/warehouse1.txt")

# ----------------------------
# Run Greedy Search
# ----------------------------
print("----- GREEDY SEARCH -----")

# Execute greedy algorithm
greedy_result = greedy_search(problem)

# Reconstruct and print path
greedy_path = reconstruct_path(greedy_result)
print("Path:", greedy_path)

# ----------------------------
# Run A* Search
# ----------------------------
print("\n----- A* SEARCH -----")

# Execute A* algorithm
astar_result = astar_search(problem)

# Reconstruct and print path
astar_path = reconstruct_path(astar_result)
print("Path:", astar_path)
```

```
----- GREEDY SEARCH -----
Path: [(1, 1), (1, 2), (1, 3), (1, 4), (1, 5), (1, 6), (1, 7), (2, 7), (3, 7),
(4, 7), (5, 7)]

----- A* SEARCH -----
Path: [(1, 1), (2, 1), (2, 2), (3, 2), (4, 2), (5, 2), (5, 3), (5, 4), (5, 5),
(5, 6), (5, 7)]
```

In [2]:
```
# This function displays the path on the grid visually
def display(grid, path):
    grid_copy = [row[:] for row in grid]

    for r, c in path:
        if grid_copy[r][c] not in ('A', 'B'):
            grid_copy[r][c] = '*'
```

```
    for row in grid_copy:
        print("".join(row))

print("\n--- Greedy Path Visualization ---")
display(problem.grid, greedy_path)

print("\n--- A* Path Visualization ---")
display(problem.grid, astar_path)
```

```
--- Greedy Path Visualization ---
##########
#A****** #
#  ####* #
#     #* #
#  ### * #
#      B #
##########

--- A* Path Visualization ---
##########
#A       #
#**####  #
# *   #  #
# *###   #
# *****B #
##########
```

Results Discussion

- Greedy Search follows the heuristic only and may not produce the shortest path.
- A* Search considers both cost and heuristic, ensuring optimality.

From the results above, we observe that A* produces a more efficient path compared to Greedy.